

Phase 4

OCR Fallback for Scanned PDFs

Tesseract · PyMuPDF rendering · 30-char trigger · DPI tradeoffs · Synthetic scanned test · Extraction-method metadata

Today's win in one line: The ingestion engine now handles scanned PDFs. When a page has no text layer, the parser silently falls back to rendering the page as an image and running Tesseract on the pixels. Verified end-to-end by rasterizing the Day 1 study guide into a text-layer-free PDF, ingesting it, and confirming all 10 pages hit the OCR branch — then retrieving the OCR'd content via `/search` at score 0.653 vs the original text-layer copy's 0.693. A ~4% quality drop from pixels → Tesseract → embedding → search. Small enough to prove OCR is competitive with direct extraction.

What you built / verified	How
Tesseract engine installed	UB-Mannheim Windows build v5.5.0 at <code>C:\Program Files\Tesseract-OCR\</code> . Path pinned in <code>.env</code> as <code>TESSERACT_CMD</code> .
Python wrapper wired	<code>pytesseract</code> + <code>Pillow</code> installed; <code>tesseract_cmd</code> pointed at the Windows binary at module import time.
OCR helper (<code>app/parsers/ocr_parser.py</code>)	Renders a PyMuPDF page to PNG bytes in memory at 200 DPI, feeds to Tesseract, returns text. No temp files on disk.
Parser upgraded	<code>extract_pdf_pages</code> now returns 3-tuples (<code>page_num</code> , <code>text</code> , <code>method</code>). Triggers OCR when text-layer content is under 30 chars.
Ingestion report enriched	New field <code>ocr_pages_count</code> exposed in JSON responses — audit trail for how many pages required OCR.
Synthetic scanned test	Built <code>tests/make_scanned_pdf.py</code> that rasterizes any PDF into a text-layer-free copy. Reused Day 1 study guide as source.
End-to-end verified	Ingested <code>Day1_SCANNED.pdf</code> : 10 pages processed, 10 chunks created, 10 OCR pages. Retrieval finds the scanned copy alongside the original at ~4% score gap.

What you built / verified	How
Committed and pushed	Local repo linked to GitHub, all 12 phases pushed to <code>origin/main</code> . Work is now off-laptop.

Companion docs: [AI Ingestion Engine Build Plan](#) · [Day 1](#) · [Day 2](#) · [Day 3 Study Guides](#)

1 Why OCR belongs in a RAG pipeline

1.1 The problem: some PDFs have no text

A PDF is a container. It *can* hold selectable text plus positioning info — that's a "digital PDF," made by exporting from Word or a browser. It can *also* hold nothing but a pixel image on each page — that's a "scanned PDF," made by a photocopier, phone scan, or someone screenshotting a book. Both look identical to a human. They are radically different to a program.

DEFINITION	A scanned PDF is one where each page is a raster image; there is no text layer to extract. PyMuPDF's <code>page.get_text()</code> returns an empty string.
WHERE SEEN	Photocopied legal contracts, invoices scanned into a filing cabinet, textbooks captured on a phone, historical archives, government form PDFs, anything a Xerox produced.
EXAMPLE	You OCR-tested this today. The synthetic scanned PDF made from Day 1 study guide returned <code>page 1 text length: 0</code> — visually identical to the original, zero readable text.
KEY TERMS	Text layer, raster page, image-only PDF, OCR (Optical Character Recognition).
WHY FOR INTERNSHIP	Any real client corpus will have some scanned documents. A RAG system that silently drops them is worse than one that refuses them — you'd never know your knowledge base has holes.

1.2 What breaks without OCR

Trace the failure through your pipeline before you had OCR:

1. `page.get_text()` returns `""` for every page.
2. The chunker gets empty strings and produces zero chunks.
3. The embedder has nothing to embed.
4. Qdrant stores nothing.
5. The service returns `chunks_created: 0`. **No exception is raised.** The API returns 201 Created like everything is fine.

The scanned file appears to have ingested successfully — the report says "1 file processed" — but it's invisible to search. This is the silent-failure mode that OCR eliminates.

THE ENGINEERING PRINCIPLE

Empty input producing empty output is technically correct behavior and operationally a disaster. Any pipeline that can silently drop content needs a check that turns silence into either an alert or a fallback. OCR is the fallback.

2 Tesseract and pytesseract

2.1 What Tesseract is

Tesseract is an open-source OCR engine. Given a page image, it detects character shapes, classifies them with a neural network trained on real fonts, groups characters into words using layout heuristics, and returns text plus (optionally) per-word confidence and bounding boxes. It's the OCR engine most Linux distributions ship by default and it's what almost every open-source pipeline uses.

DEFINITION	An OCR engine that turns pixels of text into text strings. Started at HP in the 1980s, open-sourced 2005, maintained by Google since 2006.
WHERE USED	Every layer of the modern document-processing stack: scanned-PDF search, receipt readers, screenshot → text tools, historical archive digitization.
EXAMPLE	You installed v5.5.0 today. Command-line probe: <code>tesseract --version</code> . In Python: <code>pytesseract.image_to_string(image)</code> .
KEY TERMS	OCR, character recognition, LSTM (its neural architecture), <code>tessdata</code> (language models), <code>--psm</code> (page segmentation mode).
WHY FOR INTERNSHIP	Free, mature, offline. No API keys, no rate limits, no data leaving your machine. Perfect for a company that cares about not shipping client PDFs to third-party OCR services.

2.2 pytesseract is only a wrapper

This surprised you today and it will surprise every future engineer on your team. `pip install pytesseract` installs a Python module. It does **not** install Tesseract. The Python package is a thin subprocess wrapper — it shells out to a real `tesseract.exe` binary that has to exist separately on disk.

DEFINITION	A Python wrapper that invokes the Tesseract binary via subprocess and returns its stdout.
WHERE USED	Any Python code doing OCR. LangChain's OCR loader uses it. Unstructured.io uses it. Everyone does.
EXAMPLE	On Linux/Mac, <code>apt install tesseract-ocr</code> puts <code>tesseract</code> on PATH and pytesseract finds it automatically. On Windows there's no auto-discovery, so you set <code>pytesseract.pytesseract.tesseract_cmd = "C:\\\\...\\tesseract.exe"</code> .
KEY TERMS	Subprocess, PATH, <code>tesseract_cmd</code> , config string, <code>--psm</code> .
WHY FOR INTERNSHIP	Understanding this distinction lets you debug production OCR failures. "Works on my machine" almost always means one machine had the binary installed and the other didn't.

TRAP

Hardcoding the tesseract path in code is fine until you deploy to Linux and everything breaks. You did the right thing: put the path in `.env`, keep it empty on non-Windows, and let pytesseract find PATH.

3 PyMuPDF's double role

3.1 Extractor AND renderer

You've been using PyMuPDF (`fitz`) as a text extractor since Phase 1. In Phase 4 it does something else too: it renders pages to images. That's the seam that lets OCR work without adding a second library like `pdf2image` (which needs Poppler installed separately on Windows — an install fight you avoided).

DEFINITION	PyMuPDF is a Python binding for MuPDF, a C library that parses PDFs. It can extract text, render pages as pixmaps, extract images, edit content — the full toolkit.
WHERE USED	Both stages of your parser: <code>page.get_text()</code> for the fast text-layer path, <code>page.get_pixmap(matrix=...)</code> for the OCR-fallback path.
EXAMPLE	<code>fitz.Matrix(200/72, 200/72)</code> scales the default 72-DPI render to 200 DPI. <code>pixmap.tobytes("png")</code> gives you PNG bytes without a temp file.
KEY TERMS	Pixmap, transformation matrix, DPI scaling, in-memory image bytes, PIL <code>Image.open(io.BytesIO(...))</code> .
WHY FOR INTERNSHIP	Fewer moving parts. One library covers both jobs. No Poppler install on Windows. Cross-platform code stays identical.

YOU PROVED THIS TODAY

Your synthetic scan generator uses `page.get_pixmap()` to rasterize the source PDF and `new_page.insert_image()` to re-wrap the images as a new PDF. Same library used two different ways to build the test artifact *and* handle production ingestion.

4 The 30-character trigger

4.1 How the parser decides "this needs OCR"

You want the fast path when possible — text-layer extraction is instant, OCR takes 3-8 seconds per page. So the parser tries `page.get_text()` first, then makes one decision: is what came back "content" or "empty"? The rule you shipped is `len(text.strip()) < 30`.

DEFINITION	A heuristic threshold that separates "real content" from "no content" without needing to inspect the PDF's internal structure.
WHERE USED	Once per page during PDF ingestion, right after the first <code>get_text()</code> call.
EXAMPLE	A prose page returns hundreds of chars. A pure-image scan returns 0. A page with just a page number returns 2. A page with just a chapter heading returns 15-25. 30 sits in the gap.
KEY TERMS	Heuristic, threshold, fast-path/slow-path, mixed-mode extraction.
WHY FOR INTERNSHIP	You'll get asked "how do you decide when to OCR?" Answering "I picked a threshold from measurement, not a magic number" is a professional-sounding answer.

4.2 Why not always OCR, or never OCR

Strategy	Result
Always OCR	Correct but slow. Every ingest is dominated by Tesseract time. A 100-page digital PDF that would parse in 200 ms takes 5+ minutes.
Never OCR	Fast but wrong. Scanned PDFs silently ingest as empty. This is what your pipeline did before today.
Threshold-based (what you shipped)	Fast when possible, correct when necessary. Text-layer pages take milliseconds; OCR only runs when the fast path returns nothing.

TRAP

Some PDFs are *mixed* — pages 1-10 have a text layer, pages 11-13 are scanned inserts. Your per-page threshold handles this correctly because the decision is made independently for each page. A per-document threshold would mishandle mixed docs.

5 DPI: the accuracy/size tradeoff

5.1 What DPI does to OCR

DPI = dots per inch. When you render a PDF page to an image, you pick how many pixels per inch of paper. Higher DPI = more pixels = Tesseract sees character shapes more clearly = better accuracy. But also = larger image = slower Tesseract = larger file if you save it.

DEFINITION Dots per inch — the resolution at which a page is rasterized.

WHERE USED The `matrix` arg to `page.get_pixmap()`. PDF's native resolution is 72 DPI, so `fitz.Matrix(dpi/72, dpi/72)` scales up to your target.

EXAMPLE Tesseract's own docs recommend 300 DPI for print. 200 DPI is the sweet spot for modern fonts. Below 150 accuracy drops fast.

KEY TERMS Rasterization, resolution, transformation matrix, character shape recognition.

WHY FOR INTERNSHIP Any OCR question will touch DPI. Knowing where the sweet spot is (and why) shows you've actually tuned an OCR pipeline instead of copy-pasting one.

5.2 The 110 MB blowup and what it taught you

Your first test rasterized Day 1 study guide at 200 DPI. The output file was 110 MB — for a 10-page document. Your upload validator (from Phase 0) correctly rejected it:

`PDF is 110 MB; limit is 50 MB`. You dropped DPI to 100 for the synthetic scan, hit ~27 MB, and it passed.

YOU PROVED THIS TODAY

Two independent things worked correctly at once. The upload limit fired exactly when it should have. AND OCR at 100 DPI was still good enough to embed meaningfully (0.653 similarity vs original 0.693). Both are portfolio-worthy observations.

SUBTLETY WORTH KNOWING

Your production parser still uses 200 DPI (in `ocr_parser.py`) — that's the right choice for actual OCR quality on real scanned documents. The 100 DPI was only for generating the test artifact so it would pass your upload limit. Two different DPIs for two different jobs.

6 Extraction-method metadata

6.1 Why every chunk should know how it was born

Your parser now returns 3-tuples: `(page_number, text, method)` where `method` is `"pdf_text"` or `"ocr"`. The ingestion service uses this to count OCR pages and expose the count in the report. That's the minimum you shipped. The full pattern — threading `extraction_method` into every Qdrant chunk payload — is called out in the build plan and is a Day 4/5 morning follow-up.

DEFINITION A metadata tag on each chunk (and each page) that records how the text was obtained.

WHERE USED In the ingestion report (already done). Ultimately in the Qdrant payload alongside `file_id`, `page_number`, etc. (follow-up).

EXAMPLE "This answer came from Day1_SCANNED.pdf p.5, extracted via OCR" — that's a citation an auditor can trust. If OCR quality is later found suspect, you can filter Qdrant for `extraction_method == "ocr"` and re-process just those chunks.

KEY TERMS Metadata, chunk payload, provenance, auditability.

WHY FOR INTERNSHIP Any senior engineer or auditor will ask "how do you know which chunks are trustworthy?" This field is the answer.

HONEST NOTE

Right now the field is *counted* but not *propagated* per chunk. You should say this out loud in the interview if asked, rather than oversell. "I capture the method at the parser level and report it in aggregate. Propagating it to every chunk payload is a small follow-up — one line each in the qdrant store and the ingestion service." That's a professional answer.

7 How you tested — the synthetic scanned PDF

7.1 The test artifact

You didn't have a real scanner. You didn't need one. You wrote a 30-line helper (`tests/make_scanned_pdf.py`) that opens any PDF, renders every page as a PNG, and re-wraps those PNGs as a new PDF. The output looks identical to a human, has zero selectable text, and forces the OCR branch to fire.

DEFINITION	A rasterization utility that strips the text layer from a PDF by replacing every page with an image of itself.
WHERE USED	Only in tests. Not part of the production pipeline.
EXAMPLE	<pre>python tests/make_scanned_pdf.py final_report/ Day1_Study_Guide.pdf storage/uploads/Day1_SCANNED.pdf</pre>
KEY TERMS	Test fixture, synthetic data, negative test, rasterization.
WHY FOR INTERNSHIP	Interviewers love this. "I generated my own edge-case test data" is a stronger answer than "I found a scanned PDF online." It shows you understand the input space, not just the code.

7.2 The findings

Ingest response for the scanned copy:

- `pages_processed`: 10
- `chunks_created`: 10
- `ocr_pages_count`: 10 ← **every page hit the OCR branch, exactly as expected**

Search response ("why do we use volumes in Docker"):

- Rank 1: Day1_Study_Guide.pdf p.3 (original text layer) — score **0.693**
- Rank 2: Day1_Study_Guide.pdf p.9 (original text layer) — score **0.655**
- Rank 3: **Day1_SCANNED.pdf p.2 (OCR'd copy)** — score **0.653**

YOU PROVED THIS TODAY

The scanned copy landed in the top-3 for the exact same query that worked in Phase 2, scoring within 4% of the original. Pixels → Tesseract → embedding → cosine similarity — the whole chain preserves enough semantic signal that OCR content is competitive with direct extraction.

7.3 The OCR errors you noticed (interview gold)

Read the OCR'd text closely and you can spot Tesseract's classic failure modes:

- "CL/CD" instead of "CI/CD" — capital I misread as capital L.
- "CL! -ndaemon" instead of "CLI - daemon" — character grouping and spacing errors.
- "IL1" instead of "1.1" — digit/letter confusion in section numbering.

WHY THIS MATTERS

Someone searching for "CI/CD" against the scanned corpus would miss the OCR'd chunk entirely. Real-world mitigations: post-OCR spellcheck against a domain glossary, keeping a separate keyword-search index on file+section metadata, or asking Tesseract for confidence scores and flagging low-confidence pages for re-scan. These are the answers to "what would you improve next?"

Q&A Likely interview questions on today's work

Answers in your voice — the way you'd actually explain it, not a textbook.

Question	Your answer
What is OCR and why does your app need it?	OCR turns pixels of text into actual text strings. My app needs it because some PDFs are scans — every page is just an image with no readable text layer. Without OCR my parser returns empty strings for those pages, the chunker produces zero chunks, and the scanned document silently ingests as nothing. Nothing crashes; the file just becomes invisible to search. OCR is the fallback that catches those cases.
Why not just OCR every page to be safe?	Because OCR is slow. A text-layer extraction is instant — milliseconds per page. Tesseract takes 3 to 8 seconds per page on my CPU. A 100-page digital PDF that would parse in 200 ms would take 5+ minutes if I OCR'd everything. So I try the fast path first and only fall back to OCR when it returns nothing.
How does your app decide which pages need OCR?	Simple heuristic. After extracting text normally, if the result has fewer than 30 characters after stripping whitespace, I treat the page as scanned and OCR it. A real prose page returns hundreds of chars. A pure image scan returns 0. Even a page with just a heading returns 15-25. So 30 sits in the dead zone between empty and real content — nothing legitimate falls there.

Question	Your answer
Why did you pick Tesseract?	It's free, mature, and runs offline. No API keys, no rate limits, no data leaving my machine — which matters for a company that doesn't want to ship client PDFs to a third-party service. It's also what everyone in the Python OCR ecosystem uses, so I get community support when things break.
What DPI do you render at and why?	200 DPI in the actual OCR path. That's the sweet spot for modern print — high enough for Tesseract to accurately recognize character shapes, low enough that a 10-page scan doesn't destroy CPU. Below 150 DPI accuracy drops fast; above 300 you burn cycles with no gain. Tesseract's own docs actually recommend 300, but 200 gave me clean recognition and I wanted the speed.
How did you test without a real scanner?	I wrote a helper that fakes it. It opens any PDF, renders every page to a PNG, then re-wraps those PNGs as a new PDF. The output looks identical to a human but has zero text layer — so my parser can't cheat by finding text. I used it to convert my own Day 1 study guide into a synthetic scan, ingested it, and confirmed all 10 pages hit the OCR branch.
Your first test failed with a 110 MB error. What happened?	I rasterized the source PDF at 200 DPI, which produced 10 full-resolution PNGs stored inside one PDF. It ballooned to 110 MB and my upload validator from Phase 0 rejected it — the file-size limit is 50 MB. That was actually good news: two independent things worked correctly at once. I dropped the test-fixture DPI to 100 to get under the limit. Production OCR still runs at 200 DPI, they're different jobs.
How do you know OCR extracted content that's usable?	I ran the same "why do we use volumes in Docker" query I used in Phase 2 against the collection that now has both the original AND the OCR'd copy. The OCR'd copy came back at rank 3 with a score of 0.653 versus the original's 0.693. A ~4% drop across pixels → Tesseract → embedding → cosine similarity is small enough to say OCR quality is competitive with direct extraction.
Why is the OCR'd copy's score slightly lower than the original's?	Because OCR introduces noise. My chunk from Tesseract has small errors like "CL/CD" instead of "CI/CD," "IL1" instead of "1.1," and various spacing issues. The embedding model still recognizes it as the same topic — 0.653 is a solid similarity score — but the noise costs me some semantic precision. It's not free, just cheap.

Question	Your answer
What OCR errors did you spot and what would you do about them?	Tesseract confuses visually similar characters — capital I versus capital L, digits versus letters especially in section numbers. In a Docker corpus that turns "CI/CD" into "CL/CD," which means someone searching for the correct term wouldn't find that chunk. Real deployments mitigate this with post-OCR spellcheck against a domain glossary, keeping a keyword-search index on filename and section titles as a fallback, or having Tesseract emit per-word confidence scores and flagging low-confidence pages.
Why does pytesseract need TESSERACT_CMD on Windows?	Because pytesseract is only a Python wrapper — it shells out to a real tesseract.exe binary that has to exist separately on disk. On Linux and Mac, apt or brew put tesseract on PATH and pytesseract finds it automatically. Windows doesn't do that, so I set the path once at module import time by reading TESSERACT_CMD from .env. Keeps the code cross-platform: on Linux I leave the env var empty and pytesseract finds PATH.
What's the extraction_method field for?	Provenance. Every page my parser processes gets tagged either "pdf_text" or "ocr" so I know downstream how the text was obtained. Right now I aggregate this into an ocr_pages_count on the ingest report. The next step — a small follow-up — is threading it into every chunk's Qdrant payload so I can later filter for "show me only OCR'd chunks" if I need to audit OCR quality or re-process just those pages. I chose not to do that tonight because it touches three more files and I wanted a clean commit boundary.
Walk me through what happens when someone uploads a scanned PDF.	Route validates it — content type is PDF, size under 50 MB, not corrupt. Calls the ingestion service. Service calls the parser. Parser opens the bytes with PyMuPDF, iterates through pages. On page one, get_text returns an empty string — under my 30-char threshold. So the parser calls the OCR helper, which renders that page to an in-memory PNG at 200 DPI using PyMuPDF's own pixmap, hands the bytes to Tesseract, gets text back. That text goes into the same chunker, embedder, and Qdrant upsert as any other page. The parser tags this page as extracted via "ocr" and my service counts it. The response includes ocr_pages_count so the caller knows how many pages needed OCR.
Why use PyMuPDF for rendering instead of pdf2image?	Fewer moving parts. PyMuPDF is already my text extractor from Phase 1, and it can also render pages as images via get_pixmap. Using it for both means one library covers both jobs, and I skip installing Poppler on Windows separately — which pdf2image needs. Cross-platform code stays identical.

Question	Your answer
What are the risks of your current OCR approach?	Three I'd flag. First, Tesseract's character confusion means some queries silently miss chunks — I don't currently detect that. Second, <code>extraction_method</code> isn't yet on every chunk, so a user can't tell which citations came from OCR versus text layer. Third, my 30-char threshold is heuristic — a very sparse page like a title page might get OCR'd unnecessarily. The fixes for all three are small and I'd tackle them in the polish phase.

Day 4 · Phase 4 complete. OCR fallback working end-to-end for scanned PDFs. Verified against a synthetic scan of the Day 1 study guide. Remaining phases: 5 (video ingestion), 6 (n8n workflow), 7 (polish). Roadmap items still to address in Day 4 planning: OpenCV/DeepFace CV app, ReAct agent, n8n workflows.